

Rapport de TP 1

Pile générique et analyse d'expressions mathématiques

Langage : C++ – Système : Linux

Code source : https://github.com/zakariae-bakkari/cpp_ilisi2/tree/main/tp1

Réalisé par : **Zakariae Bakkari**

28 septembre 2026

Table des matières

1	Introduction	2
2	Exercice 1 : Pile générique	2
2.1	Problématique	2
2.2	Méthodologie	3
2.2.1	Conception	3
2.2.2	Interface générique	3
2.2.3	Implémentation par tableau dynamique	4
2.2.4	Implémentation par liste chaînée	4
2.2.5	Gestion de la mémoire	5
2.2.6	Complexité	5
2.3	Résultats	5
3	Exercice 2 : Vérification d'une expression mathématique	6
3.1	Problématique	6
3.2	Méthodologie	7
3.2.1	Analyse lexicale	7
3.2.2	Automate de vérification	7
3.2.3	Généricité vis-à-vis de l'implémentation de la pile	8
3.2.4	Trace d'exécution	9
3.2.5	Évaluation : algorithme à deux piles	10
3.3	Résultats	11
3.3.1	Vérification	11
3.3.2	Évaluation	12
4	Exercice 3 : Conversion infix $\leftrightarrow$ préfixe	12
4.1	Problématique	12
4.2	Méthodologie	13
4.2.1	Infixe vers préfixe	13
4.2.2	Préfixe vers infix	13
4.3	Résultats	14
5	Bibliothèque dynamique libpile.so sous Linux	15
5.1	Problématique	15
5.2	Méthodologie	15
5.2.1	Séparation déclaration / implémentation	15
5.2.2	Construction de la bibliothèque	16
5.3	Résultats	16
5.4	Étapes de compilation	17
6	Conclusion	17
A	Compilation et exécution : le Makefile	18
A.1	Rôle d'un Makefile	18
A.2	Syntaxe d'une règle	18
A.3	Construction de la bibliothèque et des tests	18
A.4	Cible <code>etapes</code> : un fichier par étape de compilation	19
A.5	Utilisation	20

1 Introduction

La *pile* (*stack*) est une structure de données de type LIFO (*Last In, First Out*) : le dernier élément ajouté est le premier retiré. Elle est utilisée dans de nombreux algorithmes : gestion des appels de fonctions, annulation (*undo*), parcours en profondeur, et surtout **analyse syntaxique d'expressions**.

Ce TP comporte trois exercices liés entre eux :

1. concevoir une **pile générique** (**template**) avec deux implémentations : tableau dynamique et liste chaînée ;
2. utiliser cette pile pour **vérifier une expression mathématique** (validité, équilibre et ordre des délimiteurs), puis l'évaluer en respectant la priorité des opérateurs ;
3. **convertir** une expression de la notation infixe vers la notation préfixe et inversement.

Pour chaque exercice, nous présentons la **problématique**, la **méthodologie** adoptée et les **résultats** obtenus.

Environnement de travail. Le TP est réalisé sous **Linux** (Ubuntu, compilateur **g++ 15.2**, outil **make**). Le code des piles et de l'analyseur d'expressions est compilé sous forme de **bibliothèque dynamique** **libpile.so**, l'équivalent Linux d'une DLL Windows, utilisée par les trois programmes de test (section 5).

Dépôt GitHub. L'ensemble du code source, la bibliothèque, les fichiers d'étapes de compilation et ce rapport sont disponibles à l'adresse : https://github.com/zakariae-bakkari/cpp_ilisi2/tree/main/tp1.

Organisation du code.

Fichier	Rôle
include/Pile.h	interface abstraite générique Pile<T>
include/PileTableau.h, .hpp	tableau dynamique : déclaration, implémentation
include/PileListe.h, .hpp	liste chaînée : déclaration, implémentation
include/Expression.h, .hpp	analyse lexicale, vérification, évaluation, conversions
src/PileTableau.cpp	instanciations de PileTableau<T> (bibliothèque)
src/PileListe.cpp	instanciations de PileListe<T> (bibliothèque)
src/Expression.cpp	fonctions utilitaires et Analyseur (bibliothèque)
src/test_pile.cpp	tests de l'exercice 1
src/test_expression.cpp	tests de l'exercice 2
src/test_conversion.cpp	tests de l'exercice 3
Makefile	compilation de libpile.so et des tests
compilation_steps/	fichiers générés à chaque étape de compilation
guide/	guide PDF : bibliothèques .so / .a , fichiers .hpp

Pour la lisibilité, les extraits de code des sections 2 à 4 présentent les méthodes écrites à l'intérieur des classes ; dans le projet, leur code se trouve dans les fichiers **.hpp** (voir section 5).

2 Exercice 1 : Pile générique

2.1 Problématique

Une pile écrite pour un type fixe (par exemple **int**) doit être réécrite pour chaque nouveau type (**char**, **string**, **double**...), ce qui duplique le code et multiplie les erreurs. De plus, il existe plusieurs représentations possibles d'une pile, chacune avec ses avantages : il faut pouvoir en changer *sans modifier le code qui l'utilise*.

Les questions posées sont donc :

- Comment écrire une pile **indépendante du type** des éléments ?
- Comment offrir **deux représentations** (tableau dynamique et liste chaînée) derrière un **même contrat** ?
- Comment garantir une **gestion mémoire correcte** (pas de fuite, copie profonde, pile vide) ?
- Quelle représentation est la plus **performante** ?

2.2 Méthodologie

2.2.1 Conception

Nous séparons le *contrat* de l'*implémentation* :

- une classe abstraite générique `Pile<T>` déclare les opérations (`empiler`, `depiler`, `getSommet`, `estVide`, `taille`) sous forme de méthodes virtuelles pures ;
- deux classes génériques concrètes, `PileTableau<T>` et `PileListe<T>`, héritent de cette interface ;
- le code client (fonctions de test, analyseur d'expressions) ne dépend que de l'interface ou du paramètre de modèle, ce qui permet d'interchanger les implémentations.

Cette conception combine les deux formes de polymorphisme du C++ : le polymorphisme **paramétrique** (templates, choix du type `T`) et le polymorphisme **d'inclusion** (héritage et méthodes virtuelles, choix de la représentation).

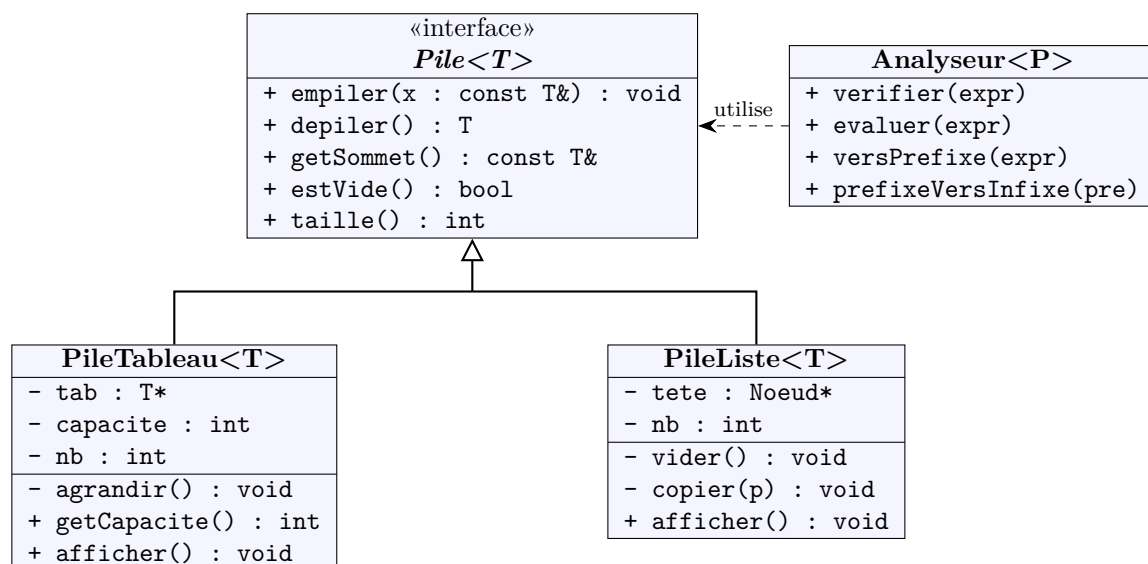


FIGURE 1 – Diagramme de classes simplifié

2.2.2 Interface générique

Listing 1 – Interface `Pile<T>` (`Pile.h`)

```

1  template <typename T>
2  class Pile {
3  public:
4      virtual ~Pile() {}
5
6      virtual void empiler(const T& x) = 0;           // ajoute x au sommet
7      virtual T depiler() = 0;                       // retire et renvoie le sommet
8      virtual const T& getSommet() const = 0;        // consulte le sommet
9      virtual bool estVide() const = 0;
10     virtual int taille() const = 0;
11 };
  
```

Le destructeur virtuel garantit que la destruction d'une pile concrète via un pointeur `Pile<T>*` libère correctement la mémoire.

2.2.3 Implémentation par tableau dynamique

Les éléments sont rangés dans un tableau alloué sur le tas ; le sommet est la case `tab[nb-1]`. Lorsque le tableau est plein, sa capacité est **doublée** : une copie en $O(n)$ n'a lieu que $\log_2 n$ fois, ce qui donne un coût **amorti** en $O(1)$ par empilement.

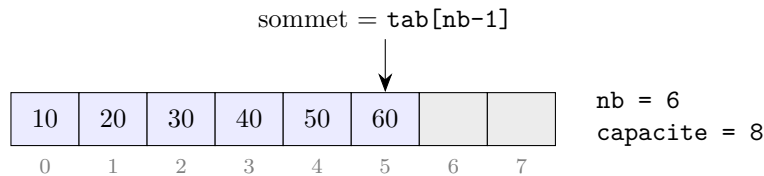


FIGURE 2 – Représentation de la pile par tableau dynamique

Listing 2 – Méthodes essentielles de `PileTableau<T>`

```

1 // double la capacité quand le tableau est plein : coût amorti O(1)
2 void agrandir() {
3     int nouvelleCapacite = capacite * 2;
4     T* nouveau = new T[nouvelleCapacite];
5     for (int i = 0; i < nb; i++)
6         nouveau[i] = tab[i];
7     delete[] tab;
8     tab = nouveau;
9     capacite = nouvelleCapacite;
10 }
11
12 void empiler(const T& x) override {
13     if (nb == capacite)
14         agrandir();
15     tab[nb++] = x;
16 }
17
18 T depiler() override {
19     if (estVide())
20         throw std::underflow_error("Pile vide");
21     return tab[--nb];
22 }

```

2.2.4 Implémentation par liste chaînée

Chaque élément est stocké dans un `Noeud` alloué individuellement ; le sommet est la **tête** de la liste, ce qui rend l'empilement et le dépilement en $O(1)$ sans jamais recopier les données.

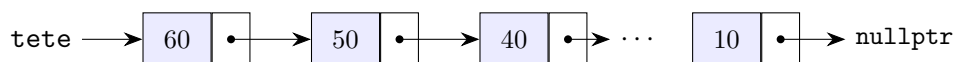


FIGURE 3 – Représentation de la pile par liste chaînée (sommet = tête)

Listing 3 – Structure et méthodes essentielles de `PileListe<T>`

```

1 struct Noeud {
2     T valeur;
3     Noeud* suivant;
4     Noeud(const T& v, Noeud* s = nullptr) : valeur(v), suivant(s) {}
5 };

```

```

6
7 void empiler(const T& x) override {
8     tete = new Noeud(x, tete);    // insertion en tête
9     nb++;
10 }
11
12 T depiler() override {
13     if (estVide())
14         throw std::underflow_error("Pile vide");
15     Noeud* tmp = tete;
16     T v = tmp->valeur;
17     tete = tete->suivant;
18     delete tmp;
19     nb--;
20     return v;
21 }

```

2.2.5 Gestion de la mémoire

Les deux classes possèdent de la mémoire dynamique; nous appliquons donc la **règle de trois** : destructeur, constructeur de copie et opérateur d'affectation effectuent une **copie profonde**. Pour `PileTableau`, l'opérateur d'affectation alloue le nouveau tableau *avant* de libérer l'ancien afin de rester cohérent si `new` échoue. Toute opération sur une pile vide lève l'exception `std::underflow_error`.

2.2.6 Complexité

Opération	PileTableau	PileListe
empiler	$O(1)$ amorti ($O(n)$ lors d'un agrandissement)	$O(1)$
depiler	$O(1)$	$O(1)$
getSommet	$O(1)$	$O(1)$
copie	$O(n)$	$O(n)$
mémoire	entre n et $2n$ cases	n nœuds (valeur + pointeur)

TABLE 1 – Comparaison théorique des deux implémentations

2.3 Résultats

Le programme `test_pile` valide chaque aspect de la conception :

Listing 4 – Extrait de la sortie de `test_pile`

```

=== 1. PileTableau<int> ===
empiler(10) -> taille=1 capacite=2
empiler(20) -> taille=2 capacite=2
empiler(30) -> taille=3 capacite=4
empiler(40) -> taille=4 capacite=4
empiler(50) -> taille=5 capacite=8
empiler(60) -> taille=6 capacite=8
contenu : [ 60 50 40 30 20 10 ]
sommet   : 60
depiler  : 60

=== 2. PileListe<string> ===
contenu : [ gamma beta alpha ]

=== 3. PileTableau<Point> (type utilisateur) ===
contenu : [ (3,4) (1,2) ]

```

```

=== 4. Polymorphisme via Pile<T>& ===
PileTableau<char>
    depilement : E L I P
PileListe<char>
    depilement : E L I P

=== 5. Copie profonde ===
original : [ 2 1 ]
copie    : [ 99 2 1 ]

=== 6. Exceptions (pile vide) ===
PileTableau : Pile vide
PileListe   : Pile vide

```

Interprétation.

- **Généricité** : la même classe fonctionne avec `int`, `string`, `char` et un type utilisateur `Point`.
- **Agrandissement** : la capacité double (2, 4, 8) exactement lorsque le tableau est plein.
- **Polymorphisme** : la fonction `viderEtAfficher(Pile<T>&)` traite les deux implémentations de façon identique; empiler « PILE » puis tout dépiler donne bien « ELIP » (ordre LIFO).
- **Copie profonde** : modifier la copie ne modifie pas l'original.
- **Robustesse** : le dépilement d'une pile vide est intercepté par une exception.

Performances. Temps mesurés pour empiler puis dépiler n entiers (compilation `-O2`) :

n	PileTableau (ms)	PileListe (ms)	Rapport
10^5	0,30	3,03	$\approx 10\times$
10^6	2,14	36,18	$\approx 17\times$
10^7	32,24	320,12	$\approx 10\times$

TABLE 2 – Comparaison expérimentale des deux implémentations

Les deux implémentations ont un temps linéaire en n , mais le tableau dynamique est environ **dix fois plus rapide**. La liste effectue une allocation (`new`) et une libération (`delete`) par élément, tandis que le tableau ne réalloue que $\log_2 n$ fois (≈ 22 fois pour $n = 10^7$) et stocke ses éléments de façon contiguë, ce qui exploite le cache du processeur. La liste reste intéressante lorsque l'on veut éviter les recopies (éléments coûteux à copier) ou garantir un temps constant *dans le pire cas* pour chaque opération.

3 Exercice 2 : Vérification d'une expression mathématique

3.1 Problématique

Avant d'évaluer une expression comme $\{ [(1+2) \times 3] - 4 \} / 5$, il faut s'assurer qu'elle est **bien formée**. Plusieurs types d'erreurs doivent être détectés :

1. **caractères interdits** : `3 # 4` ;
2. **délimiteurs non équilibrés** : `(3 + 4, 3 + 4)` ;
3. **délimiteurs mal appariés** : `(3 + 4]` ;
4. **ordre des délimiteurs non respecté** : par convention mathématique, les accolades contiennent les crochets qui contiennent les parenthèses, $\{ [()] \}$; ainsi `( [ 1 + 2 ] )` est refusé ;

5. **erreurs de syntaxe** entre opérandes et opérateurs : $3 + * 4$, $3 4 + 5$, $(3 +)$, $()$, $3 +$.

Une fois l'expression validée, il faut pouvoir l'**évaluer** en respectant la **priorité** ($\wedge > * / > + -$) et l'**associativité** des opérateurs. La difficulté est que l'imbrication des délimiteurs est arbitrairement profonde : un simple compteur ne suffit pas à vérifier l'appariement, il faut *mémoriser l'ordre* des ouvertures, d'où l'usage d'une pile.

3.2 Méthodologie

3.2.1 Analyse lexicale

La fonction **decouper** transforme la chaîne en une suite de **jetons** (**Token**) : nombre (entier ou décimal), variable (identificateur), opérateur ($+ - * / \wedge$), délimiteur ouvrant ($([\{$) ou fermant ($)] \}$). Chaque jeton mémorise sa **position** dans la chaîne afin de localiser précisément les erreurs. Les espaces sont ignorés et tout autre caractère déclenche une erreur.

3.2.2 Automate de vérification

La vérification combine deux mécanismes :

- un **automate à deux états** qui contrôle l'alternance opérande/opérateur ;
- une **pile générique de jetons** ($P<Token>$) qui mémorise les délimiteurs ouverts et non encore fermés.

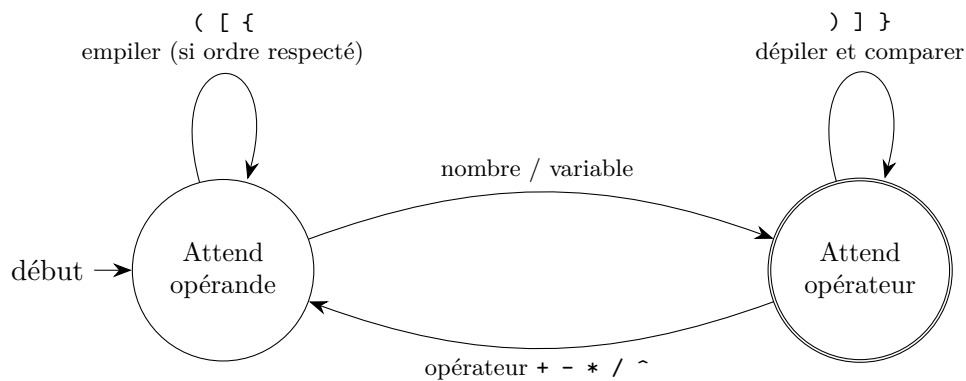


FIGURE 4 – Automate de vérification. Toute autre transition est une erreur ; l'expression est acceptée si l'on termine dans l'état « attend opérateur » avec une pile vide.

Le rang d'un délimiteur est défini par $\text{rang}(() = 1$, $\text{rang}([] = 2$, $\text{rang}(\{ \} = 3$. Un délimiteur ouvrant n'est accepté que si son rang est **inférieur ou égal** à celui du sommet de la pile : on peut écrire $((a))$ ou $\{[(a)]\}$, mais pas $([a])$.

Algorithme 1 Vérification d'une expression infixe**Entrée :** suite de jetons $t_1 \dots t_k$ **Sortie :** valide, ou message d'erreur et position

```

1 : pile ← pile vide; attendOperande ← vrai
2 : pour tout jeton t faire
3 :     si t est un nombre ou une variable alors
4 :         si non attendOperande alors retourner erreur « opérateur manquant »
5 :         fin si
6 :         attendOperande ← faux
7 :     sinon si t est un opérateur alors
8 :         si attendOperande alors retourner erreur « opérande manquant »
9 :         fin si
10 :        attendOperande ← vrai
11 :    sinon si t est un délimiteur ouvrant alors
12 :        si non attendOperande alors retourner erreur « opérateur manquant »
13 :        fin si
14 :        si pile non vide et rang(t) > rang(sommet(pile)) alors retourner erreur d'ordre
15 :        fin si
16 :        empiler(pile, t)
17 :    sinon ▷ délimiteur fermant
18 :        si pile vide alors retourner erreur « fermant sans ouvrant »
19 :        fin si
20 :        si attendOperande alors retourner erreur « opérande manquant »
21 :        fin si
22 :        o ← dépiler(pile)
23 :        si o n'est pas l'ouvrant associé à t alors retourner erreur d'appariement
24 :        fin si
25 :    fin si
26 : fin pour
27 : si pile non vide alors retourner erreur « délimiteur jamais fermé »
28 : fin si
29 : si attendOperande alors retourner erreur « expression incomplète »
30 : fin si
31 : retourner valide

```

3.2.3 Généricité vis-à-vis de l'implémentation de la pile

L'analyseur est une classe paramétrée par la *classe de pile* elle-même (paramètre de modèle de modèle) : `Analyseur<PileTableau>` et `Analyseur<PileListe>` exécutent le même algorithme avec une représentation différente.

Listing 5 – Cœur de la méthode `verifier`

```

1  template <template <typename> class P>
2  class Analyseur {
3  public:
4      static Resultat verifier(const std::string& expr) {
5          // ... découpage en jetons, expression vide ...
6          P<Token> delimites;
7          bool attendOperande = true;    // état de l'automate
8
9          for (const Token& t : tokens) {
10             switch (t.type) {
11                 case NOMBRE:
12                 case VARIABLE:
13                     if (!attendOperande)

```

```

14         return {false, "Operande '" + t.texte + "' inattendu (
15             operateur manquant)", t.position};
16         attendOperande = false;
17         break;
18     case OPERATEUR:
19         if (attendOperande)
20             return {false, "Operateur '" + t.texte + "' inattendu (
21                 operande manquant)", t.position};
22         attendOperande = true;
23         break;
24     case OUVRANTE:
25         if (!attendOperande)
26             return {false, "'" + t.texte + "' inattendu (operateur
27                 manquant)", t.position};
28         if (!delimiteurs.estVide()
29             && rangDelimiteur(t.texte[0]) > rangDelimiteur(delimiteurs
30                 .getSommet().texte[0]))
31             return {false, "Ordre non respecte : ...", t.position};
32         delimiteurs.empiler(t);
33         break;
34     case FERMANTE: {
35         if (delimiteurs.estVide())
36             return {false, "'" + t.texte + "' sans delimiteur ouvrant"
37                 , t.position};
38         if (attendOperande)
39             return {false, "Operande manquant avant '" + t.texte + "'"
40                 , t.position};
41         Token ouvrant = delimiteurs.depiler();
42         if (ouvrant.texte[0] != ouvranteAssociee(t.texte[0]))
43             return {false, "'" + t.texte + "' ne ferme pas '" +
44                 ouvrant.texte + "'", t.position};
45         break;
46     }
47 }
48 };

```

3.2.4 Trace d'exécution

Pour l'expression { [(1 + 2) * 3] - 4 } / 5 (sommet de la pile à droite; O = attend opérande, P = attend opérateur) :

Étape	Jeton	Action	Pile	État
1	{	empiler	{	O
2	[	rang $2 \leq 3$: empiler	{ [	O
3	(	rang $1 \leq 2$: empiler	{ [(	O
4	1	opérande	{ [(	P
5	+	opérateur	{ [(	O
6	2	opérande	{ [(	P
7	)	dépiler (: apparié	{ [	P
8	*	opérateur	{ [	O
9	3	opérande	{ [	P
10	]	dépiler [: apparié	{	P
11	-	opérateur	{	O
12	4	opérande	{	P
13	}	dépiler { : apparié	(vide)	P
14	/	opérateur	(vide)	O
15	5	opérande	(vide)	P

Fin : pile vide et état P $\Rightarrow$ expression **valide**

TABLE 3 – Trace de la vérification

3.2.5 Évaluation : algorithme à deux piles

Pour évaluer une expression valide, nous utilisons l'algorithme à deux piles de Dijkstra : une pile d'**opérateurs** (P<char>) et une pile de **valeurs** (P<V>).

- un opérande est empilé sur la pile des valeurs ;
- un délimiteur ouvrant est empilé sur la pile des opérateurs ;
- un délimiteur fermant déclenche des *réductions* jusqu'à l'ouvrant correspondant ;
- un opérateur *op* déclenche des réductions tant que le sommet est un opérateur de priorité *supérieure*, ou *égale* si *op* est associatif à gauche, puis *op* est empilé ;
- une **réduction** dépile un opérateur et deux valeurs *a, b* et empile `combiner(op, a, b)`.

L'algorithme est écrit **une seule fois**, de façon générique : le type des valeurs *V* et la fonction `combiner` sont des paramètres. Avec *V* = `double` et `combiner` = calcul, on obtient l'**évaluation** ; avec *V* = `string` et `combiner` = concaténation, on obtient les **conversions** de l'exercice 3.

Listing 6 – Parcours générique d'une expression infixe

```

1  template <typename V>
2  static V parcourirInfixe(const std::string& expr,
3                          std::function<V(const Token&)> feuille,
4                          std::function<V(char, const V&, const V&)> combiner) {
5      Resultat r = verifier(expr);
6      if (!r.valide)
7          throw ErreurExpression(r.message, r.position);
8
9      P<char> operateurs;
10     P<V> valeurs;
11
12     // dépile un opérateur et ses deux opérandes, empile le résultat
13     auto reduire = [&]() {
14         char op = operateurs.depiler();
15         V b = valeurs.depiler();
16         V a = valeurs.depiler();
17         valeurs.empiler(combiner(op, a, b));
18     };
19
20     for (const Token& t : decouper(expr)) {
21         switch (t.type) {
22             case NOMBRE:
23             case VARIABLE:

```

```

24         valeurs.empiler(feuille(t));
25         break;
26     case OUVRANTE:
27         operateurs.empiler(t.texte[0]);
28         break;
29     case FERMANTE:
30         while (!estOuvrante(opérateurs.getSommet()))
31             reduire();
32         operateurs.depiler(); // retire le délimiteur ouvrant
33         break;
34     case OPERATEUR: {
35         char op = t.texte[0];
36         while (!opérateurs.estVide() && estOperateur(opérateurs.getSommet())) {
37             char haut = opérateurs.getSommet();
38             if (priorite(haut) > priorite(op)
39                 || (priorite(haut) == priorite(op) && !associatifDroite(op)))
40                 reduire();
41             else
42                 break;
43         }
44         operateurs.empiler(op);
45         break;
46     }
47 }
48 }
49 while (!opérateurs.estVide())
50     reduire();
51 return valeurs.depiler();
52 }
53
54 static double evaluer(const std::string& expr) {
55     return parcourirInfixe<double>(expr,
56         [](const Token& t) -> double {
57             if (t.type == VARIABLE)
58                 throw std::invalid_argument("Variable '" + t.texte + "' non
59                     evaluable");
60             return std::stod(t.texte);
61         },
62         [](char op, const double& a, const double& b) { return appliquer(op, a, b)
63             ; });
64 }

```

3.3 Résultats

3.3.1 Vérification

Le programme `test_expression` soumet 20 expressions à l'analyseur. Chaque erreur est localisée par un curseur `^` :

Listing 7 – Extrait de la sortie de `test_expression`

```

"{ [ ( 1 + 2 ] ) }"
      ^
-> INVALIDE (pos 12) : ']' ne ferme pas '(' (position 4)
"([ 1 + 2 ] )"
      ^
-> INVALIDE (pos 2) : Ordre non respecte : '[' a l'interieur de '('
"3 + * 4"
      ^
-> INVALIDE (pos 4) : Operateur '*' inattendu (operande manquant)

```

Expression	Verdict	Diagnostic (position)
3 + 4 * 2	valide	–
(a + b) * c	valide	–
{ [(1 + 2) * 3] - 4 } / 5	valide	–
[(x - y) * (x + y)] ^ 2	valide	–
((2))	valide	–
(3 + 4	invalide	(jamais fermé (0)
3 + 4)	invalide	) sans délimiteur ouvrant (5)
(3 + 4]	invalide	] ne ferme pas ((6)
{ [(1 + 2]) }	invalide	] ne ferme pas ((12)
([1 + 2])	invalide	ordre : [à l'intérieur de ((2)
[{ a }]	invalide	ordre : { à l'intérieur de [(2)
3 + * 4	invalide	opérateur * inattendu (4)
3 4 + 5	invalide	opérande 4 inattendu (2)
(3 +)	invalide	opérande manquant avant) (5)
()	invalide	opérande manquant avant) (1)
2 (3 + 4)	invalide	(inattendu, opérateur manquant (2)
* 3	invalide	opérateur * inattendu (0)
3 +	invalide	expression incomplète (2)
3 # 4	invalide	caractère non autorisé # (2)
(chaîne vide)	invalide	expression vide (0)

TABLE 4 – Résultats de la vérification (identiques avec `PileTableau` et `PileListe` : 0 différence)

Toutes les catégories d'erreurs de la problématique sont détectées, avec un message explicite et la position exacte du jeton fautif. Les deux implémentations de la pile produisent des verdicts identiques, ce qui confirme qu'elles respectent le même contrat.

3.3.2 Évaluation

Expression	Résultat	Propriété vérifiée
3 + 4 * 2	11	priorité de $\times$ sur +
(3 + 4) * 2	14	parenthèses
{ [(1 + 2) * 3] - 4 } / 5	1	délimiteurs imbriqués
10 - 4 - 3	3	associativité à gauche
100 / 10 / 5	2	associativité à gauche
2 ^ 3 ^ 2	512	associativité à droite : $2^{(3^2)}$
2.5 * 4	10	nombres décimaux
1 / (2 - 2)	erreur	division par zéro
x + 1	erreur	variable non évaluable

TABLE 5 – Résultats de l'évaluation

4 Exercice 3 : Conversion infixe $\leftrightarrow$ préfixe

4.1 Problématique

En notation **infixe** ($a + b$), l'opérateur est placé entre ses opérandes : elle est naturelle pour l'humain mais nécessite des parenthèses et des règles de priorité. En notation **préfixe** ou polonaise ($+ a b$), l'opérateur précède ses opérandes : l'expression est **non ambiguë sans parenthèses** et s'évalue simplement avec une pile. Le problème est de passer d'une notation à l'autre :

- **infixe** → **préfixe** en respectant la priorité et l’associativité ($a - b - c \rightarrow - - a b c$ mais $a \wedge b \wedge c \rightarrow \wedge a \wedge b c$);
- **préfixe** → **infixe** en réintroduisant les parenthèses nécessaires, et en détectant les expressions préfixes mal formées.

4.2 Méthodologie

4.2.1 Infixe vers préfixe

Nous réutilisons directement le parcours générique à deux piles de l’exercice 2, avec $V = \text{string}$: chaque réduction construit la chaîne « $\text{op } a \text{ } b$ ». La conversion vers la notation postfixe s’obtient de la même façon avec « $a \text{ } b \text{ } \text{op}$ ».

Listing 8 – Conversions obtenues par spécialisation du parcours générique

```

1 // Infixe -> préfixe (notation polonaise) : "op a b"
2 static std::string versPrefixe(const std::string& expr) {
3     return parcourirInfixe<std::string>(expr,
4         [](const Token& t) { return t.texte; },
5         [](char op, const std::string& a, const std::string& b) {
6             return std::string(1, op) + " " + a + " " + b;
7         });
8 }
9
10 // Infixe -> postfixe (notation polonaise inverse) : "a b op"
11 static std::string versPostfixe(const std::string& expr) {
12     return parcourirInfixe<std::string>(expr,
13         [](const Token& t) { return t.texte; },
14         [](char op, const std::string& a, const std::string& b) {
15             return a + " " + b + " " + std::string(1, op);
16         });
17 }

```

Jeton	Action	Pile opérateurs	Pile valeurs
a	empiler valeur	–	a
+	pile vide : empiler	+	a
b	empiler valeur	+	a b
*	priorité(+) < priorité(*) : empiler	+ *	a b
c	empiler valeur	+ *	a b c
fin	réduire *	+	a * b c
fin	réduire +	–	+ a * b c

TABLE 6 – Trace de la conversion de $a + b * c$ en préfixe

4.2.2 Préfixe vers infixe

Une expression préfixe se lit **de droite à gauche** avec une seule pile :

- un opérande est empilé;
- un opérateur dépile son premier opérande a puis son second b , et empile « $(a \text{ } \text{op } b)$ »;
- à la fin, la pile doit contenir exactement un élément; sinon l’expression est mal formée (opérandes ou opérateurs manquants).

Ce parcours est lui aussi générique : avec $V = \text{double}$, il **évalue** directement l’expression préfixe.

Listing 9 – Parcours générique d’une expression préfixe

```

1 template <typename V>
2 static V parcourirPrefixe(const std::string& prefixe,
3                             std::function<V(const std::string&)> feuille,
4                             std::function<V(char, const V&, const V&)> combiner) {

```

```

5 // ... découpage de la chaîne en jetons séparés par des espaces ...
6 P<V> valeurs;
7 for (int i = (int)jetons.size() - 1; i >= 0; i--) {
8     const std::string& s = jetons[i];
9     if (s.size() == 1 && estOperateur(s[0])) {
10         if (valeurs.taille() < 2)
11             throw std::invalid_argument("Operandes manquants pour '" + s + "'");
12         V a = valeurs.depiler(); // premier opérande
13         V b = valeurs.depiler(); // second opérande
14         valeurs.empiler(combiner(s[0], a, b));
15     } else if (estNombre(s) || estIdentificateur(s)) {
16         valeurs.empiler(feuille(s));
17     } else {
18         throw std::invalid_argument("Jeton invalide '" + s + "'");
19     }
20 }
21 if (valeurs.taille() != 1)
22     throw std::invalid_argument("Trop d'operandes (operateur manquant)");
23 return valeurs.depiler();
24 }
25
26 // Préfixe -> infixe (entièrement parenthésée)
27 static std::string prefixeVersInfixe(const std::string& prefixe) {
28     std::string s = parcourirPrefixe<std::string>(prefixe,
29         [](const std::string& o) { return o; },
30         [](char op, const std::string& a, const std::string& b) {
31             return "(" + a + " " + std::string(1, op) + " " + b + " ";
32         });
33     // les parenthèses extérieures sont superflues
34     if (s.size() > 1 && s.front() == '(' && s.back() == ')')
35         s = s.substr(1, s.size() - 2);
36     return s;
37 }

```

Jeton (droite → gauche)	Action	Pile (sommet à droite)
c	empiler	c
b	empiler	c b
a	empiler	c b a
+	$a \leftarrow a, b \leftarrow b$	c (a + b)
*	$a \leftarrow (a + b), b \leftarrow c$	((a + b) * c)

TABLE 7 – Trace de la conversion de * + a b c en infixe : résultat (a + b) * c

4.3 Résultats

Infixe	Préfixe	Postfixe
a + b	+ a b	a b +
a + b * c	+ a * b c	a b c * +
(a + b) * c	* + a b c	a b + c *
a - b - c	- - a b c	a b - c -
a ^ b ^ c	^ a ^ b c	a b c ^ ^
{ [(a + b) * c] - d } / e	/ - * + a b c d e	a b + c * d - e /
(x - y) * (x + y) ^ 2	* - x y ^ + x y 2	x y - x y + 2 ^ *

TABLE 8 – Conversion infixe → préfixe et postfixe

Préfixe	Infixe obtenue
+ a b	a + b
* + a b c	(a + b) * c
- - a b c	(a - b) - c
- a - b c	a - (b - c)
/ - * + a b c d e	((a + b) * c) - d) / e
+ a	erreur : opérandes manquants pour +
+ a b c	erreur : trop d'opérandes (opérateur manquant)
+ a \$	erreur : jeton invalide \$

TABLE 9 – Conversion préfixe → infixe

Vérification croisée. Pour valider les conversions, chaque expression numérique est évaluée de trois manières : directement en infixe, via sa forme préfixe, et après un aller-retour infixe → préfixe → infixe. Les trois valeurs doivent coïncider.

Listing 10 – Extrait de la sortie de `test_conversion`

```

2 ^ 3 ^ 2
  prefixe = ^ 2 ^ 3 2
  retour  = 2 ^ (3 ^ 2)
  valeurs = 512 / 512 / 512   [OK]
(8 - 2) * (8 + 2) ^ 2
  prefixe = * - 8 2 ^ + 8 2 2
  retour  = (8 - 2) * ((8 + 2) ^ 2)
  valeurs = 600 / 600 / 600   [OK]

6/6 tests reussis

```

Les six tests croisés réussissent : la conversion préserve la sémantique de l'expression, y compris la priorité des opérateurs et l'associativité à gauche (−, /) comme à droite (^). Les expressions préfixes mal formées sont rejetées avec un message explicite.

5 Bibliothèque dynamique `libpile.so` sous Linux

5.1 Problématique

Les piles et l'analyseur doivent être fournis sous forme de **bibliothèque dynamique** : du code compilé une seule fois, chargé au lancement par tous les programmes qui l'utilisent. Sous Windows, il s'agit d'une DLL (`.dll`) ; sous Linux, système utilisé pour ce TP, d'un *shared object* (`.so`).

La difficulté vient des **templates**. `PileTableau<T>` n'est qu'un modèle : le compilateur ne produit du code machine que lorsqu'il est utilisé avec un type précis (`PileTableau<int>`), c'est l'*instanciation*. Dans la première version du TP, tout le code était dans les en-têtes (*header-only*) : une bibliothèque compilée à partir de ces fichiers aurait été **vide**, puisqu'aucun type n'y est instancié.

5.2 Méthodologie

5.2.1 Séparation déclaration / implémentation

Chaque classe générique est répartie sur trois fichiers :

Fichier	Contenu
<code>PileTableau.h</code>	déclaration de la classe et liste des types déjà compilés dans la bibliothèque (extern template)
<code>PileTableau.hpp</code>	code des méthodes du template, écrit hors de la classe
<code>PileTableau.cpp</code>	instanciation explicite pour les types utilisés, compilé dans <code>libpile.so</code>

L'extension `.tpp` (*template implementation*) est une convention : c'est un fichier destiné à être **inclus**, jamais compilé seul, ce qui le distingue d'un `.cpp`.

Listing 11 – Instanciation explicite (`src/PileTableau.cpp`, compilé dans la bibliothèque)

```
1 #include "PileTableau.tpp"
2
3 template class PileTableau<int>;           // génère ici tout le code de
    PileTableau<int>
4 template class PileTableau<double>;
5 template class PileTableau<char>;
6 template class PileTableau<std::string>;
```

Listing 12 – Déclaration vue par les programmes clients (fin de `include/PileTableau.h`)

```
1 // Instanciations fournies par libpile.so : le compilateur ne les régénère
2 // pas dans le programme client, il utilise celles de la bibliothèque.
3 extern template class PileTableau<int>;
4 extern template class PileTableau<double>;
5 extern template class PileTableau<char>;
6 extern template class PileTableau<std::string>;
```

Le même principe s'applique à `PileListe<T>` et à l'analyseur : `src/Expression.cpp` contient les fonctions utilitaires (`decouper`, `appliquer`, `priorite...`) et instancie `PileTableau<Token>`, `PileListe<Token>`, `Analyseur<PileTableau>` et `Analyseur<PileListe>`.

Une bibliothèque ne peut pas connaître tous les types possibles. Pour le type utilisateur `Point` de l'exercice 1, `test_pile.cpp` inclut `PileTableau.tpp` : `PileTableau<Point>` est alors généré dans le programme lui-même, tandis que `PileTableau<int>` provient toujours de la bibliothèque.

5.2.2 Construction de la bibliothèque

Listing 13 – Étapes de construction (automatisées par le `Makefile`)

```
# 1. objets en code independant de la position (-fPIC)
g++ -Wall -Wextra -O2 -Iinclude -fPIC -c src/PileTableau.cpp -o obj/PileTableau.o
g++ -Wall -Wextra -O2 -Iinclude -fPIC -c src/PileListe.cpp -o obj/PileListe.o
g++ -Wall -Wextra -O2 -Iinclude -fPIC -c src/Expression.cpp -o obj/Expression.o

# 2. regroupement des objets dans la bibliotheque partagee
g++ -shared -Wl,-soname,libpile.so obj/*.o -o libpile.so

# 3. edition de liens d'un programme client avec libpile.so
g++ -Wall -Wextra -O2 -Iinclude src/test_pile.cpp -L. -lpile -Wl,-rpath,'$ORIGIN' -o test_pile
```

- `-fPIC` : une `.so` est chargée à une adresse différente à chaque exécution, son code ne doit pas dépendre d'une adresse fixe ;
- `-shared` : produit une bibliothèque partagée au lieu d'un exécutable ;
- `-L. -lpile` : cherche `libpile.so` dans le dossier courant ;
- `-Wl,-rpath,'$ORIGIN'` : enregistre dans l'exécutable qu'il doit chercher la bibliothèque dans son propre dossier, sans avoir à définir `LD_LIBRARY_PATH`.

5.3 Résultats

Les trois programmes de test donnent exactement les mêmes sorties qu'avec la version *header-only*. Les outils Linux confirment qu'ils utilisent bien la bibliothèque :

Listing 14 – Vérification de la liaison dynamique

```
$ ldd test_pile | grep pile
    libpile.so => ../tp1/libpile.so

$ readelf -d test_expression | grep -E 'NEEDED|RUNPATH'
```

```
(NEEDED) Shared library: [libpile.so]
(RUNPATH) Library runpath: [$ORIGIN]

$ nm -DC --undefined-only test_expression | grep Analyseur
U Analyseur<PileTableau>::verifier(std::string const&)
U Analyseur<PileListe>::evaluer(std::string const&)
U Analyseur<PileListe>::verifier(std::string const&)
```

Le symbole U (*undefined*) indique que `test_expression` ne contient pas le code de l'analyseur : il est résolu au lancement dans `libpile.so`, qui exporte 336 symboles de piles et d'analyseur.

Comparaison avec une bibliothèque statique. Les mêmes objets peuvent être archivés en bibliothèque statique (`ar rcs libpile.a obj/*.o`), dont le code est *copié* dans l'exécutable à l'édition de liens.

	Dynamique (libpile.so)	Statique (libpile.a)
Taille de <code>test_expression</code>	27 Ko	175 Ko
Bibliothèque absente au lancement	erreur : cannot open shared object file	fonctionne
Mise à jour de la bibliothèque	remplacer le <code>.so</code> suffit	refaire l'édition de liens
Mémoire	une copie partagée entre les programmes	une copie par programme

TABLE 10 – Bibliothèque dynamique et statique (mesures sur le TP)

5.4 Étapes de compilation

Le dossier `compilation_steps/` contient un programme de démonstration (includes, macros, compilation conditionnelle, commentaires, espaces) qui utilise `libpile.so`. Son `Makefile` exécute chaque étape séparément et conserve le fichier produit :

Étape	Commande	Fichier produit	Observation
prétraitement	<code>g++ -E</code>	<code>programme.ii</code>	37 023 lignes, macros remplacées
compilation	<code>g++ -S</code>	<code>programme.s</code>	code assembleur x86-64
assemblage	<code>g++ -c</code>	<code>programme.o</code>	méthodes de la pile non définies (U)
édition de liens	<code>g++ -lpile</code>	<code>programme</code>	lié à <code>libpile.so</code> (ldd)

TABLE 11 – Fichiers générés dans `compilation_steps/`

Après le prétraitement, les commentaires ont disparu, les `#include` sont remplacés par le contenu des en-têtes (d'où les 37 023 lignes, dues surtout à `<iostream>`), `CARRE(i)` devient `((i) * (i))` et le bloc `#ifdef DEBUG` est supprimé. Le fichier objet `programme.o` référence `PileTableau<int>::empiler` sans le définir : c'est l'édition de liens qui le relie à `libpile.so`.

6 Conclusion

Ce TP a permis de concevoir une **pile générique** réutilisable et de montrer son intérêt sur un problème concret d'analyse d'expressions.

- La séparation entre l'interface `Pile<T>` et ses deux implémentations permet de changer de représentation sans toucher au code client : les analyseurs `Analyseur<PileTableau>` et `Analyseur<PileListe>` donnent des résultats identiques.
- Le **tableau dynamique** est environ dix fois plus rapide en pratique grâce à la contiguïté mémoire et au doublement de capacité ; la **liste chaînée** garantit un $O(1)$ strict par opération.

- La pile rend la vérification des délimiteurs simple et linéaire ($O(n)$) ; couplée à un automate à deux états, elle détecte toutes les erreurs de syntaxe et les localise précisément.
- Un unique algorithme générique à deux piles, paramétré par le type des valeurs et une fonction de combinaison, réalise à la fois l'évaluation et les conversions infixe $\rightarrow$ préfixe/postfixe.
- Grâce à l'instanciation explicite des templates et à la séparation `.h` / `.tpp`, le code est distribué sous forme de bibliothèque dynamique `libpile.so` sous Linux : les programmes clients sont plus petits et profitent d'une mise à jour de la bibliothèque sans être recompilés.

Limites et perspectives. Les opérateurs unaires (par exemple -3) et les fonctions (`sin`, `max`...) ne sont pas pris en charge ; ils pourraient être ajoutés en enrichissant l'automate d'un état et la table des priorités. L'évaluation pourrait également accepter un environnement associant une valeur à chaque variable.

A Compilation et exécution : le Makefile

A.1 Rôle d'un Makefile

Un **Makefile** est un fichier de **recettes de compilation** lu par la commande `make`. Il ne remplace pas l'exécutable : il décrit *comment le fabriquer*. Sans lui, construire le TP demanderait de taper à chaque modification une dizaine de commandes `g++` longues, dans le bon ordre et sans oublier d'option (`-fPIC`, `-rpath`...).

Son intérêt principal est la **compilation incrémentale** : `make` compare la date de modification de chaque fichier avec celle de ses dépendances et ne relance que les commandes nécessaires. Si seul `src/test_pile.cpp` est modifié, la bibliothèque n'est pas recompilée ; si un en-tête change, tout ce qui en dépend est reconstruit.

A.2 Syntaxe d'une règle

Listing 15 – Forme générale d'une règle

```
cible: dependances
    commande qui fabrique la cible a partir des dependances
```

- la **cible** est le fichier à produire (ou un nom d'action comme `clean`) ;
- les **dépendances** sont les fichiers nécessaires : si l'une d'elles est plus récente que la cible, la commande est relancée ;
- la **commande** (la « recette ») doit commencer par une **tabulation**, pas par des espaces.

Le Makefile du TP utilise aussi des **variables** (`CXX = g++`, utilisée sous la forme `$(CXX)`), des **règles génériques** avec `%` (une seule règle pour tous les fichiers `src/%.cpp`) et des **variables automatiques** :

Variable	Signification
<code>\$@</code>	la cible de la règle
<code>\$<</code>	la première dépendance
<code>\$^</code>	toutes les dépendances
<code> obj</code>	dépendance « d'ordre » : le dossier doit exister, sa date est ignorée
<code>.PHONY</code>	cibles qui ne sont pas des fichiers (<code>all</code> , <code>run</code> , <code>clean</code> ...)

A.3 Construction de la bibliothèque et des tests

Listing 16 – Makefile (1/2) : bibliotheque et programmes de test

```
CXX      = g++
CXXFLAGS = -Wall -Wextra -O2 -Iinclude
HEADERS  = $(wildcard include/*.h include/*.tpp)
```

```

PROGS    = test_pile test_expression test_conversion

# Bibliotheque partagee (equivalent Linux d'une DLL)
LIB      = libpile.so
LIB_OBJS = obj/PileTableau.o obj/PileListe.o obj/Expression.o

all: $(LIB) $(PROGS)

# -fPIC : code independant de la position, obligatoire dans une .so
obj/%.o: src/%.cpp $(HEADERS) | obj
    $(CXX) $(CXXFLAGS) -fPIC -c $< -o $@

$(LIB): $(LIB_OBJS)
    $(CXX) -shared -Wl,-soname,$(LIB) $^ -o $@

# programmes lies a libpile.so, cherchee dans leur propre dossier
%: src/%.cpp $(HEADERS) $(LIB)
    $(CXX) $(CXXFLAGS) $< -L. -lpile -Wl,-rpath,'$$ORIGIN' -o $@

obj:
    mkdir -p obj

run: all
    ./test_pile && ./test_expression && ./test_conversion

```

Élément	Rôle
CXX, CXXFLAGS	compilateur et options communes : avertissements (-Wall -Wextra), optimisation (-O2), dossier des en-têtes (-Iinclude)
HEADERS	liste des .h et .hpp (fonction wildcard) : modifier un en-tête déclenche la recompilation de ce qui en dépend
all	cible par défaut (lancée par make seul) : la bibliothèque puis les trois programmes
obj/%.o	compile chaque src/X.cpp de la bibliothèque en obj/X.o, avec -fPIC
\$(LIB)	regroupe les trois objets dans libpile.so (-shared)
% (programmes)	compile src/test_X.cpp en exécutable lié à libpile.so; \$\$ORIGIN est écrit \$\$ car \$ est un caractère spécial pour make
obj	crée le dossier des fichiers objets
run	construit tout puis exécute les trois séries de tests

A.4 Cible etapes : un fichier par étape de compilation

Listing 17 – Makefile (2/2) : etapes de compilation et nettoyage

```

ETAPES = compilation

etapes: $(ETAPES)/test_pile.ii $(ETAPES)/test_pile.s $(ETAPES)/test_pile.o \
    $(ETAPES)/libpile.a $(ETAPES)/libpile.so \
    $(ETAPES)/test_pile $(ETAPES)/test_pile_statique

# 1. pretraitement : includes recopies, macros remplacees, commentaires supprimes
$(ETAPES)/%.ii: src/%.cpp $(HEADERS) | $(ETAPES)
    $(CXX) $(CXXFLAGS) -E $< -o $@

# 2. compilation : C++ pretraite -> assembleur x86-64
$(ETAPES)/%.s: $(ETAPES)/%.ii
    $(CXX) $(CXXFLAGS) -S $< -o $@

# 3. assemblage : assembleur -> code objet (non executable)
$(ETAPES)/%.o: $(ETAPES)/%.s
    $(CXX) -c $< -o $@

# 4. bibliotheques : archive statique (.a) et bibliotheque partagee (.so)

```

```

$(ETAPES)/libpile.a: $(LIB_OBJS) | $(ETAPES)
    ar rcs $@ $^

$(ETAPES)/libpile.so: $(LIB) | $(ETAPES)
    cp $< $@

# 5. edition de liens : dynamique (libpile.so chargée au lancement)
#    et statique (code de libpile.a copié dans l'exécutable)
$(ETAPES)/test_pile: $(ETAPES)/test_pile.o $(ETAPES)/libpile.so
    $(CXX) $< -L$(ETAPES) -lpile -Wl,-rpath,'$$ORIGIN' -o $@

$(ETAPES)/test_pile_statique: $(ETAPES)/test_pile.o $(ETAPES)/libpile.a
    $(CXX) $^ -o $@

$(ETAPES):
    mkdir -p $(ETAPES)

clean:
    rm -rf obj $(LIB) $(PROGS) $(ETAPES)

.PHONY: all run etapes clean

```

La cible `etapes` enchaîne les étapes de la compilation de `test_pile.cpp` en conservant le fichier produit par chacune dans le dossier `compilation/`. Chaque règle prend en entrée le fichier de l'étape précédente : `make` déduit seul l'ordre d'exécution.

Fichier	Commande	Taille	Contenu
test_pile.ii	g++ -E	1 084 Ko	code prétraité : 42 160 lignes (en-têtes recopiés, macros remplacées)
test_pile.s	g++ -S	48 Ko	assembleur x86-64 (2 159 lignes)
test_pile.o	g++ -c	29 Ko	code objet ; méthodes de la pile non définies
libpile.a	ar rcs	328 Ko	bibliothèque statique (archive des trois .o)
libpile.so	g++ -shared	210 Ko	bibliothèque dynamique
test_pile	g++ -lpile	29 Ko	exécutable lié dynamiquement : charge <code>libpile.so</code> au lancement
test_pile_statique	g++ libpile.a	58 Ko	exécutable autonome : le code utile de <code>libpile.a</code> est copié dedans

TABLE 12 – Fichiers générés par `make etapes`

L'exécutable statique est deux fois plus gros que l'exécutable dynamique, mais bien plus petit que `libpile.a` : l'éditeur de liens ne copie que les fichiers objets de l'archive dont le programme a besoin (les piles, pas l'analyseur d'expressions).

A.5 Utilisation

```

$ make      # compile libpile.so puis les trois programmes
$ make run  # execute les trois series de tests
$ make etapes  # genere un fichier par etape dans compilation/
$ make clean  # supprime tous les fichiers generes

```